

Name: Snuthi Rithika - P

Reg no: 192521407

course: compiler design

course code: CSA 1406

C02 AT-3

1) Expression Ambiguity & Right Associativity

Given Grammar:

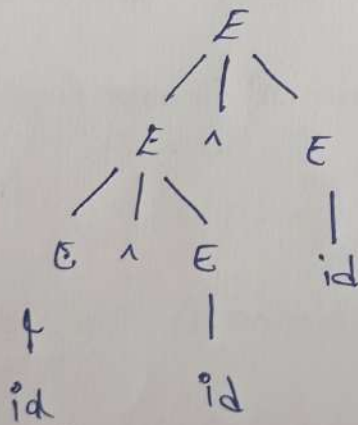
$$E \rightarrow E \wedge E / id$$

Input

$id \wedge id \wedge id$

i) show ambiguity using parse trees

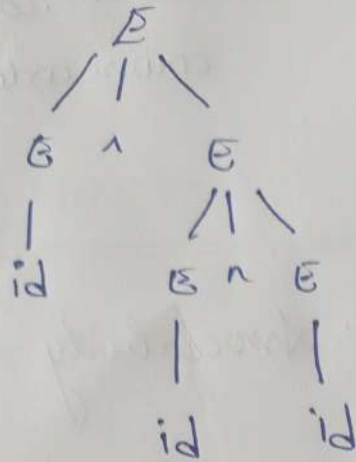
Parse Tree 1 (left Associative)



Expression:

$(id \wedge id) \wedge id$

Parse Tree 2 (Right Associative)



Expression :

$id \wedge (id \wedge id)$

ii) Define Right Associativity.

Means that when multiple operators of the same precedence appear together, evaluation starts from right to left.

Exa: $a \wedge b \wedge c$

iii) Modify the grammar

$E \rightarrow id R$

$R \rightarrow \wedge E | \epsilon$

or equivalently,

$E \rightarrow id$

$E \rightarrow id \wedge E$

iv) Demonstrate parsing

Input:

$id \wedge id \wedge id$

Modified grammar:

E

$\Rightarrow id \wedge E$

$\Rightarrow id \wedge id \wedge E$

$\Rightarrow id \wedge id \wedge id$

Grouping becomes

$id \wedge (id \wedge id)$

only one parse tree is possible.

v) Compare evaluation.

Ambiguous Grammar

$id \wedge (id \wedge id)$

Modified Grammar

$id \wedge (id \wedge id)$

2) Error Recovery in parsing

Input:

$id ++ id + id$

Grammar:

$E \rightarrow E + T / T$

$T \rightarrow id$

i) Identify error positions:

valid expression:

$id + id + id$

Received:

$id ++id + id$

Error:

$id + +id + id$

↑

Parser error message:

syntax error:

unexpected '+' after '+'.

Expected Identifier.

ii) Apply Panic mode Recovery.

* Read $id +$

* Encounter unexpected '+'

* Skip the extra '+'

* Resume parsing from id

Recovered Input:

$id + id + id$

iv) Explain synchronization tokens.

Common synchronization tokens include:

.
:
)
{

End of input (\$))

Benefits:

Prevents repeated errors

continues compilation

Reports multiple syntax errors

v) suggest improvements

* Use panic mode recovery with synchronization tokens.

* Apply phrase-level error correction for small mistake

* Display clear error messages with line and token numbers.

* continue parsing to detect multiple errors.

3) streaming Parsing in an IoT system

Input :

$\langle \langle \text{data} \rangle \text{data} \rangle$

Grammar :

$s \rightarrow \langle s \rangle s \mid \text{data}$

i) Explain streaming parsing

characteristic :

- * Handles continuous sensor input
- * Parses incrementally
- * uses minimal memory.
- * suitable for real-time IoT applications
- * Detect errors immediately.

ii) Demonstrate parsing

Input:

$\langle \langle \text{data} \rangle \text{data} \rangle$

Derivation :

$$\begin{aligned} & s \\ \Rightarrow & \langle s \rangle s \\ \Rightarrow & \langle \langle s \rangle s \rangle s \\ \Rightarrow & \langle \langle \text{data} \rangle s \rangle s \\ \Rightarrow & \langle \langle \text{data} \rangle \text{data} \rangle s \\ \Rightarrow & \langle \langle \text{data} \rangle \text{data} \rangle \end{aligned}$$

iii)

Handle incomplete input

Example of partial arrival:

Received:

<< data

The parser:

- * store the current parsing state

- * waits for additional input.

- * Does not report an error immediately.

> data >

iv)

Detect errors

Example:

< data data >

Parser message:

Syntax Error:

unexpected 'data'

Expected '>' before 'data'

Error:

A missing < or > cause the grammar to fail

v) Recommended Improvements :

- * Use incremental (streaming) parsing for continuous data.

- * Buffer incomplete input until enough tokens are available.

- * Maintain parser state between data arrivals.

- * Provide immediate error detection and recovery.